



Oracle Database Administration

I

Chapter 11: Large Objects

محتويات الفصل:

Objectives	2
1. Describing LOB Types.....	3
2. Illustrating LOB Locators, Indexes, and Chunks	5
3. Distinguishing Between BasicFiles and SecureFiles.....	7
4. Creating a Table with a LOB Column	9
5. Maintaining LOB Columns.....	15
6. Measuring LOB Space Consumed	20
7. Reading BFILEs.....	24
8. Quiz.....	25
References	31

Objectives:

1. LOB Types
2. Illustrating LOB Locators, Indexes, and Chunks
3. Table with a LOB Column
4. Measuring LOB Space Consumed

1. Describing LOB Types:

LOBs are a data type that is suited to storing large and unstructured data, such as text, log, image, video, sound, and spatial data. A LOB can hold up to a maximum size ranging from 8 terabytes to 128 terabytes depending on how your database is configured. Storing data in LOBs enables you to access and manipulate the data efficiently in your application.

Oracle supports the following types of LOBs:

Data Type	Description	Maximum Size
CLOB	Character large object for storing character documents, such as big text files, log files, XML files, and so on	(4GB-1) * block size
NCLOB	National character large object; stores data in national character set format; supports characters with varying widths	(4GB-1) * block size
BLOB	Binary large object for storing unstructured bitstream data (images, video, and so on). they participate in database transactions and can be backed up, restored, and recovered by Oracle	(4GB-1) * block size
BFILE	Binary file large object stored on the filesystem outside of database; read-only; are not covered by any Oracle security, backup and recovery, replication, or disaster recovery mechanisms.	2 ⁶⁴ -1 bytes (OS may impose a size limit that is less than this)

Using LOBs for Semi-Structured Data

Examples of semi-structured data include document files such as json, XML documents or word processor, and log files. These kinds of documents contain data in a logical structure that is processed or interpreted by an application, and is not broken down into smaller logical units when stored in the database.

Applications involving semi-structured data typically use large amounts of character data. The Character Large Object (CLOB) and National Character Large Object (NCLOB) data types are ideal for storing and manipulating this kind of data.

Binary File objects (data types) can also store character data. You can use BFILEs to load read-only data from operating system files into CLOB or NCLOB instances that you then manipulate in your application.

Using LOBs for Unstructured Data

Unstructured data cannot be decomposed into standard components. For example, data about an employee can be structured into a name, which is stored as a string; an identifier, such as an ID number, a salary and so on. A photograph, on the other hand, consists of a long stream of 1s and 0s. These bits are used to switch pixels on or off so that you can see the picture on a display, but are not broken down into any finer structure for database storage.

Also, unstructured data such as text, graphic images, still video clips, full motion video, and sound waveforms tends to be large in size. A typical employee record may be a few hundred bytes, while even small amounts of multimedia data can be thousands of times larger.

SQL data types that are ideal for large amounts of unstructured binary data include the BLOB data type (Binary Large Object) and the BFILE data type (Binary File object)

.

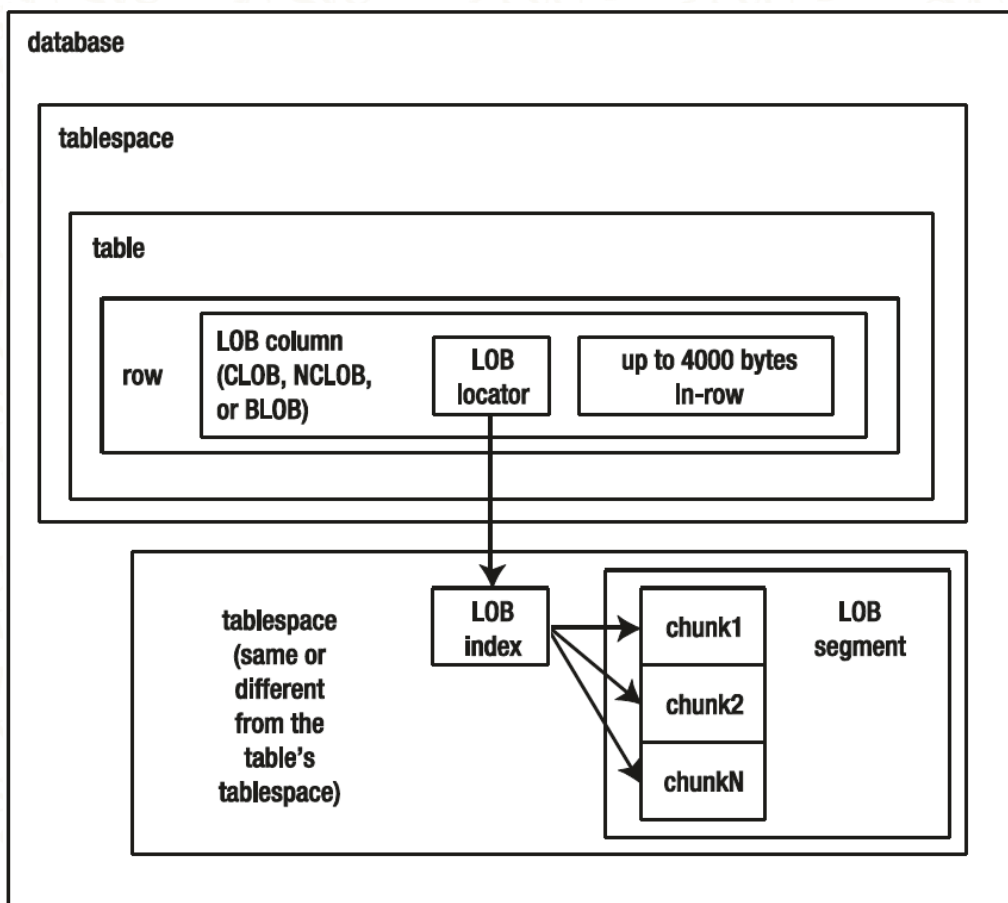
2. Illustrating LOB Locators, Indexes, and Chunks:

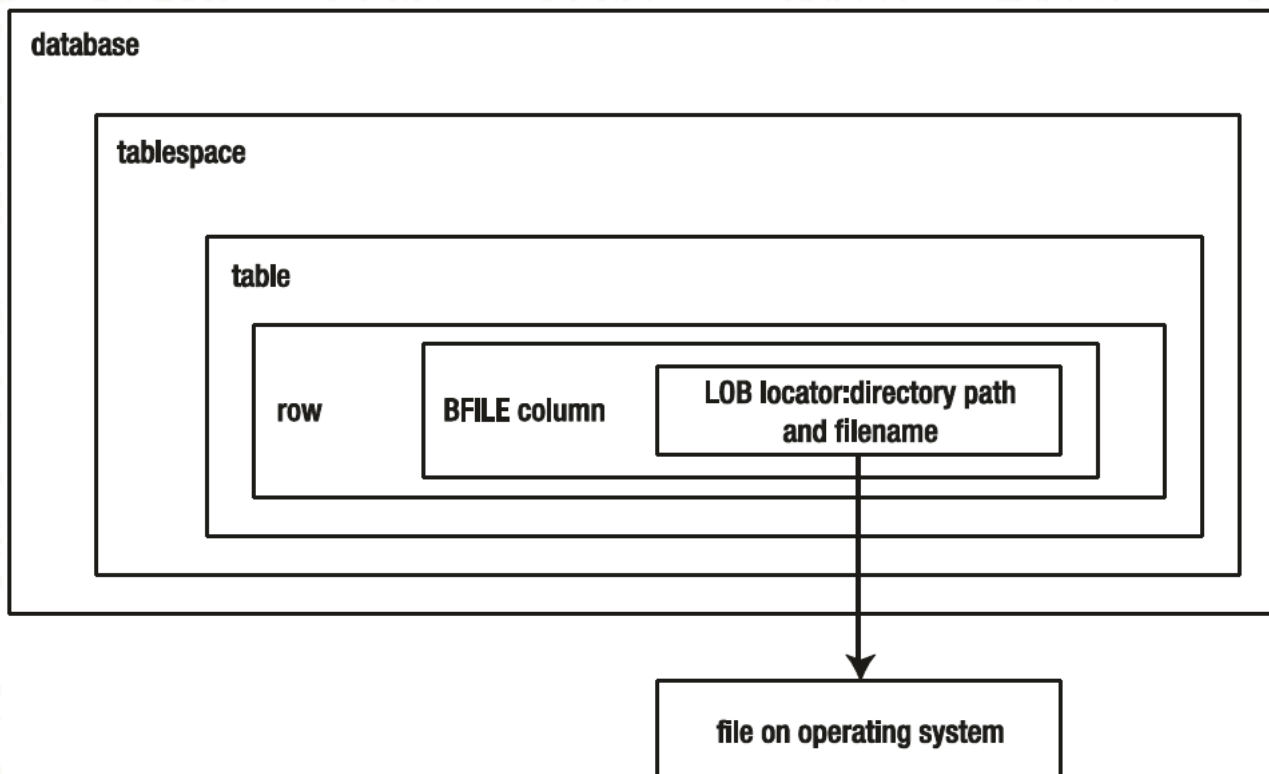
Internal LOBs (CLOB, NCLOB, BLOB) store data in pieces called chunks. A chunk is the smallest unit of allocation for a LOB and is made up of one or more database blocks. LOB locators are stored in rows containing a LOB column. The LOB locator points to a LOB index. The LOB index stores information regarding the location of LOB chunks. When a table is queried, the database uses the LOB locator and associated LOB index to locate the appropriate LOB chunks.

Please check [figure1](#) to see the relationship between a table, a row, a LOB locator, and a LOB locator's associated index and chunks:

In External LOBs; The LOB locator for a BFILE stores the directory path and file name on the OS. Please check [figure2](#) to see a BFILE LOB locator that references a file on the OS:

Note: The DBMS_LOB package performs operations on LOBs through the LOB locator.





3. Distinguishing Between BasicFiles and SecureFiles:

Several significant improvements were made to LOBs. Oracle now distinguishes between two different types of underlying LOB architecture:

Basic Files

Basic Files is the name Oracle gives to the LOB architecture available prior to Oracle Database 11g. It is still important to understand the Basic Files LOBs because many shops use Oracle versions that do not support Secure Files. Be aware that in Oracle Database 11g, the default type of LOB is still Basic Files. However, now, the default type of LOB is now Secure Files and should be used as the way to store LOBs.

Secure Files

Secure Files is the recommended option to use with the LOB architecture. It includes the following enhancements (over Basic Files LOBs):

- Encryption (requires Oracle Advanced Security option)
- Compression (requires Oracle Advanced Compression option)
- Deduplication (requires Oracle Advanced Compression option)

Secure Files encryption lets you transparently encrypt LOB data (just like other data types). The compression feature allows for significant space savings. The deduplication feature eliminates duplicate LOBs that otherwise would be stored multiple times. You need to do a small amount of planning before using Secure Files. Specifically, use of Secure Files requires the following:

- A Secure Files LOB must be stored in a tablespace, using ASSM.
- The `DB_SECUREFILE` initialization parameter controls whether a Secure Files file can be used and also defines the default LOB architecture for your database.

A Secure Files LOB must be created within a tablespace using ASSM. To create an ASSM-enabled tablespace, specify the `SEGMENT SPACE MANAGEMENT AUTO` clause; for example,

```
SQL> create tablespace lob_data  
datafile '/u01/dbfile/o18c/lob_data01.dbf'
```



```
size 1000m
extent management local
uniform size 1m
segment space management auto;
```

If you have existing tablespaces, you can verify the use of ASSM by querying the `DBA_TABLESPACES` view. The `SEGMENT_SPACE_MANAGEMENT` column should have a value of `AUTO` for any tablespaces that you want to use with Secure Files:

```
select tablespace_name, segment_space_management
from dba_tablespaces;
```

Also, Secure Files usage is governed by the `DB_SECUREFILE` database parameter. You can use either `ALTER SYSTEM` or `ALTER SESSION` to modify the value of `DB_SECUREFILE`. The following table describes the valid values for `DB_SECUREFILE`:

DB_SECUREFILE Setting	Description
NEVER	Creates the LOB as a BasicFiles type, regardless of whether the <code>SECUREFILE</code> option is specified
PERMITTED	Allows creation of SecureFiles LOBs
PREFERRED	Default value; specifies that all LOBs are created as a SecureFiles type, unless otherwise stated
ALWAYS	Creates the LOB as a SecureFiles type, unless the underlying tablespace is not using ASSM
IGNORE	Ignores the SecureFiles option, along with any SecureFiles settings

4. Creating a Table with a LOB Column:

The default underlying LOB architecture is Secure Files. It is recommended to create a LOB as a Secure Files. As discussed previously, Secure Files allows you to use features such as compression and encryption.

Creating a Basic Files LOB Column

To create a LOB column, you have to specify a LOB data type. It is best to explicitly specify the `STORE AS BASICFILE` clause in order to avoid confusion as to which LOB architecture is implemented. Listed next is such an example:

```
SQL> create table patchmain(  
patch_id number  
,patch_desc clob)  
tablespace users  
lob(patch_desc) store as basicfile;
```

When you create a table with a LOB column, you must be aware of some technical underpinnings. Review the following list, and be sure you understand each point:

- Prior to Oracle Database 12c, LOBs, by default, are created as the Basic Files type
- Oracle creates a LOB segment and a LOB index for each LOB column
- The LOB segment has a name of this format: `SYS_LOB<string>`
- The LOB index has a name of this format: `SYS_IL<string>`
- The `<string>` is the same for each LOB segment and its associated index
- The LOB segment and index are created in the same tablespace as the table, unless you specify a different tablespace
- A LOB segment and a LOB index are not created until a record is inserted into the table (the so-called deferred segment creation feature). This means that `DBA/ALL/USER_SEGMENTS` and `DBA/ALL/USER_EXTENTS` have no information in them until a row is inserted into the table

Oracle creates a LOB segment and a LOB index for each LOB column. The LOB segment stores the data. The LOB index keeps track of where the chunks of LOB data are physically stored and in what order they should be accessed.

You can query the DBA/ALL/USER_LOBS view to display the LOB segment and LOB index names:

```
SQL> select table_name, segment_name, index_name, securefile,
in_row from user_lobs;
```

Here is the output for this example:

TABLE_NAME	SEGMENT_NAME	INDEX_NAME	SEC	IN_
-----	-----	-----	---	---
PATCHMAIN	SYS_LOB0000022332C00002\$\$	SYS_IL0000022332C00002\$\$	NO	YES

You can also query DBA/USER/ALL_SEGMENTS to view information regarding LOB segments. As mentioned earlier, an initial segment is not created until you insert a row into the table (deferred segment creation). This can be confusing because you may expect a row to be present in DBA/ALL/USER_SEGMENTS immediately after you create the table:

```
SQL> select segment_name, segment_type, segment_subtype,
bytes/1024/1024
meg_bytes
from user_segments
where segment_name IN ('&&table_just_created',
'&&lob_segment_just_created',
'&&lob_index_just_created');
```

The prior query prompts for the segment names. The output shows no rows:

no rows selected

Next, insert a record into the table that contains the LOB column:

```
SQL> insert into patchmain values(1,'clob text');
```

Rerunning the query against `USER_SEGMENTS` shows that three segments have been created—one for the table, one for the LOB segment, and one for the LOB index:

SEGMENT NAME	SEGMENT TYPE	SEGMENT SU	MEG BYTES
PATCHMAIN	TABLE	ASSM	.0625
SYS_IL0000022332C00002\$\$	LOBINDEX	ASSM	.0625
SYS_LOB0000022332C00002\$\$	LOBSEGMENT	ASSM	.0625

Implementing a LOB in a Specific Tablespace

By default, the LOB segment is stored in the same tablespace as its table. You can specify a separate tablespace for a LOB segment by using the `LOB...STORE AS` clause of the `CREATE TABLE` statement. The next table creation script creates the table in a tablespace and creates separate tablespaces for the CLOB and BLOB columns:

```
SQL> create table patchmain
(patch_id number
,patch_desc clob
,patch blob
) tablespace users
lob (patch_desc) store as (tablespace lob_data)
,lob (patch) store as (tablespace lob_data);
```

The following query verifies the associated tablespaces for this table:

```
SQL> select table_name, tablespace_name, 'N/A' column_name
from user_tables
where table_name='PATCHMAIN'
union
select table_name, tablespace_name, column_name
from user_lobs
where table_name='PATCHMAIN';
```

Here is the output:

TABLE_NAME	TABLESPACE_NAME	COLUMN_NAME
PATCHMAIN	LOB_DATA	PATCH

PATCHMAIN
PATCHMAIN

LOB_DATA
USERS

PATCH_DESC
N/A

If you think the LOB segment will require different storage characteristics (such as size and growth patterns), then it is recommended that you create the LOB in a separated tablespace from that of the table data. This allows you to manage the LOB column storage separately from the regular table data storage.

Creating a Secure Files LOB Column

As discussed previously, the default LOB architecture is Secure Files. Having said that, I recommend that you explicitly state which LOB architecture to implement in order to avoid any confusion. As mentioned earlier, the tablespace that contains the Secure File LOB must be ASSM managed. Here is an example that creates a Secure Files LOB:

```
SQL> create table patchmain(
patch_id number
,patch_desc clob)
lob(patch_desc) store as securefile (tablespace lob_data);
```

Before viewing the data dictionary details regarding the LOB column, insert a record into the table to ensure that segment information is available (owing to the deferred segment allocation feature in Oracle Database 11g Release 2 and higher); for example:

```
SQL> insert into patchmain values(1,'clob text');
```

You can now verify a LOB's architecture by querying the USER_SEGMENTS view:

```
SQL> select segment_name, segment_type, segment_subtype
from user_segments;
```

Here is some sample output, indicating that a LOB segment is a Secure Files type:

SEGMENT_NAME	SEGMENT_TYPE	SEGMENT_SUBTYPE
PATCHMAIN	TABLE	ASSM
SYS_IL0000022340C00002\$\$	LOBINDEX	ASSM
SYS_LOB0000022340C00002\$\$	LOBSEGMENT	SECUREFILE

You can also query the USER_LOBS view to verify the Secure Files LOB architecture:

```
SQL> select table_name, segment_name, index_name, securefile,
in_row from user_lobs;
```

Here is the output:

TABLE_NAME	SEGMENT_NAME	INDEX_NAME	SEC	IN
PATCHMAIN	SYS_LOB0000022340C00002\$\$	SYS_IL0000022340C00002\$\$	YES	YES

Note: With the SecureFiles architecture, you no longer need to specify the following options: CHUNK, PCTVERSION, FREEPOOLS, FREELIST, and FREELIST GROUPS.

Implementing a Partitioned LOB

You can create a partitioned table that has a LOB column. Doing so lets you spread a LOB across multiple tablespaces. Such partitioning helps with balancing I/O, maintenance, and backup and recovery operations.

You can partition LOBs by RANGE, LIST, or HASH. The next example creates a LIST– Partitioned table in which LOB column data are stored in tablespaces separate from those of the table data:

```
SQL> CREATE TABLE patchmain(
patch_id NUMBER
,region VARCHAR2(16)
,patch_desc CLOB)
LOB(patch_desc) STORE AS (TABLESPACE patch1)
PARTITION BY LIST (REGION) (
PARTITION p1 VALUES ('EAST')
LOB(patch_desc) STORE AS SECUREFILE
(TABLESPACE patch1 COMPRESS HIGH)
TABLESPACE inv_data1,
PARTITION p2 VALUES ('WEST')
LOB(patch_desc) STORE AS SECUREFILE
(TABLESPACE patch2 DEDUPLICATE NOCOMPRESS)
TABLESPACE inv_data2,
PARTITION p3 VALUES (DEFAULT)
LOB(patch_desc) STORE AS SECUREFILE
(TABLESPACE patch3 COMPRESS LOW)
TABLESPACE inv_data3
```


);

Note that each LOB partition is created with its own storage options. You can view the details about the LOB partitions as shown:

```
SQL> select table_name, column_name, partition_name,
tablespace_name,compression, deduplication
from user_lob_partitions;
```

Here is some sample output:

TABLE_NAM	COLUMN_NAM	PARTITION	TABLESPACE_NAM	COMPR	DEDUPLICATIO
E	E	—	E	E	N
-----	-----	-----	-----	-----	-----
-	-	-	-	--	-
PATCHMAIN	PATCH_DESC	P1	PATCH1	HIGH	NO
PATCHMAIN	PATCH_DESC	P2	PATCH2	NO	LOB
PATCHMAIN	PATCH_DESC	P3	PATCH3	LOW	NO

Note: You can also view DBA/ALL_USER_PART_LOBS for information about partitioned LOBs.

You can change the storage characteristics of a partitioned LOB column after it is created. To do so, use the ALTER TABLE ... MODIFY PARTITION statement.

This example alters a LOB partition to have a high degree of compression:

```
SQL> alter table patchmain modify partition p2
lob (patch_desc) (compress high);
```

The next example modifies a partitioned LOB not to keep duplicate values (via the DEDUPLICATE clause):

```
SQL> alter table patchmain modify partition p3
lob (patch_desc) (deduplicate lob);
```

Note: Partitioning and Advanced Compression, which have been discussed in this chapter, are extra cost options that are available only with the Oracle Enterprise Edition.

5. Maintaining LOB Columns:

Some common maintenance tasks that are performed on LOB columns or that otherwise involve LOB columns, including moving columns between tablespaces and adding new LOB columns to a table.

Moving a LOB Column

If you create a table with a LOB column and do not specify a tablespace, then, by default, the LOB is created in the same tablespace as its table. This happens sometimes in environments in which the DBAs do not plan ahead very well; only after the LOB column has consumed large amounts of disk space does the DBA wonder why the table has grown so big. You can use the `ALTER TABLE...MOVE...STORE AS` statement to move a LOB column to a tablespace separate from that of the table.

Here is the basic syntax:

```
SQL> alter table <table_name> move lob(<lob_name>) store as  
(tablespace <new_tablespace>);
```

The next example moves the LOB column to the `LOB_DATA` tablespace:

```
SQL> alter table patchmain  
move lob(patch_desc)  
store as securefile (tablespace lob_data);
```

You can verify that the LOB was moved by querying `USER_LOBS`:

```
SQL> select table_name, column_name, tablespace_name from  
user_lobs;
```

To summarize, if the LOB column is populated with large amounts of data, you almost always want to store the LOB in a tablespace separate from that of the rest of the table data. In these scenarios, the LOB data have different growth and storage requirements and are best maintained in their own tablespace.

Adding a LOB Column

If you have an existing table to which you want to add a LOB column, use the `ALTER TABLE...ADD` statement. The next statement adds the `INV_IMAGE` column to a table:

```
SQL> alter table patchmain add(inv_image blob);
```

This statement is fine for quickly adding a LOB column to a development environment. For anything else, you should specify the storage characteristics. For instance, this command specifies that a SecureFiles LOB be created in the `LOB_DATA` tablespace:

```
SQL> alter table patchmain add(inv_image blob)
lob(inv_image) store as securefile(tablespace lob_data);
```

Removing a LOB Column

You may have a scenario in which your business requirements change, and you no longer need a column. Before you remove a column, consider renaming it so that you can better identify whether any applications or users are still accessing it:

```
SQL> alter table patchmain rename column patch_desc to
patch_desc_old;
```

After you determine that nobody is using the column, use the `ALTER TABLE...DROP` statement to drop it:

```
SQL> alter table patchmain drop(patch_desc_old);
```

You can also remove a LOB column by dropping and re-creating a table (without the LOB column). This, of course, permanently removes any data as well. Also keep in mind that if your recycle bin is enabled, then when you do not drop a table with the `PURGE` clause, space is still consumed by the dropped table. If you want to remove the space associated with the table, use the `PURGE` clause, or purge the recycle bin after dropping the table.

Caching LOBs

By default, when reading and writing LOB columns, Oracle does not cache LOBs in memory. You can change the default behavior by setting the cache-related storage options. This example specifies that Oracle should cache a LOB column in memory:

```
SQL> create table patchmain(
patch_id number
,patch_desc clob)
lob(patch_desc) store as (tablespace lob_data cache);
```

You can verify the LOB caching with this query:

```
SQL> select table_name, column_name, cache from user_lobs;
```

Here is some sample output:

TABLE_NAME	COLUMN_NAME	CACHE
PATCHMAIN	PATCH_DESC	YES

If you have LOBs that are frequently read and written to, consider using the `CACHE` option. If your LOB column is read frequently but rarely written to, then the `CACHE READS` setting is more appropriate. If the LOB column is infrequently read or written to, then the `NOCACHE` setting is suitable.

Storing LOBs In- and Out of Line

By default, up to approximately 4,000 characters of a LOB column are stored inline with the table row. If the LOB is more than 4,000 characters, then Oracle automatically stores it outside the row data. The main advantage of storing a LOB in row is that small LOBs (fewer than 4,000 characters) require less I/O, because Oracle does not have to search out of row for the LOB data.

However, storing LOB data in row is not always desirable. The disadvantage of storing LOBs in row is that the table row sizes are potentially longer. This can affect the performance of full-table scans, range scans, and updates to columns other than the LOB column.

In these situations, you may want to disable storage in the row. For example, you explicitly instruct Oracle to store the LOB outside the row with the `DISABLE STORAGE IN ROW` clause:

```
SQL> create table patchmain(  
  patch_id number  
  ,patch_desc clob  
  ,log_file blob)  
  lob(patch_desc, log_file)  
  store as (  
    tablespace lob_data  
    disable storage in row);
```

If you want to store up to 4,000 characters of a LOB in the table row, use the `ENABLE STORAGE IN ROW` clause when creating the table:

```
SQL> create table patchmain(  
  patch_id number  
  ,patch_desc clob  
  ,log_file blob)  
  lob(patch_desc, log_file)  
  store as (  
    tablespace lob_data  
    enable storage in row);
```

Note: The LOB locator is always stored inline with the row.

You cannot modify the LOB storage in a row after the table has been created. The only ways to alter storage in row are to move the LOB column or drop and re-create the table. This example alters the storage in row by moving the LOB column:

```
SQL> alter table patchmain  
  move lob(patch_desc)  
  store as (enable storage in row);
```

You can verify the in-row storage via the `IN_ROW` column of `USER_LOBS`:

```
SQL> select table_name, column_name, tablespace_name, in_row  
  from user_lobs;
```

A value of YES indicates that the LOB is stored in row:

TABLE_NAME	COLUMN_NAME	TABLESPACE_NAME	IN_ROW
-----	-----	-----	-----
PATCHMAIN	LOG_FILE	LOB_DATA	YES
PATCHMAIN	PATCH_DESC	LOB_DATA	YES

6. Measuring LOB Space Consumed:

As discussed previously, a LOB consists of an in-row lob locator, a LOB index, and a LOB segment that is made up of one or more chunks. The space used by the LOB index is usually negligible compared with the space used by the LOB segment. You can view the space consumed by a segment by querying the `BYTES` column of `DBA/ALL/USER_SEGMENTS` (just like any other segment in the database).

Example:

```
SQL> select segment_name, segment_type, segment_subtype,  
bytes/1024/1024 meg_bytes  
from user_segments;
```

You can modify the query to report on only LOBs by joining to the `USER_LOBS` view:

```
SQL> select a.table_name, a.column_name, a.segment_name,  
a.index_name  
,b.bytes/1024/1024 meg_bytes  
from user_lobs a, user_segments b  
where a.segment_name = b.segment_name;
```

You can also use the `DBMS_SPACE.SPACE_USAGE` package and procedure to report on the blocks being used by a LOB. This package only works on objects that have been created in an ASSM-managed tablespace.

There are two different forms of the `SPACE_USAGE` procedure: one form reports on Basic Files LOBs, and the other reports on Secure Files LOBs.

Basic Files Space Used

Here is an example of how to call `DBMS_SPACE.SPACE_USAGE` for a Basic Files LOB:

```
SQL> declare  
p_fs1_bytes number;  
p_fs2_bytes number;
```

```
p_fs3_bytes number;
p_fs4_bytes number;
p_fs1_blocks number;
p_fs2_blocks number;
p_fs3_blocks number;
p_fs4_blocks number;
p_full_bytes number;
p_full_blocks number;
p_unformatted_bytes number;
p_unformatted_blocks number;
begin
dbms_space.space_usage(
segment_owner => user,
segment_name => 'SYS_LOB0000024082C00002$$',
segment_type => 'LOB',
fs1_bytes => p_fs1_bytes,
fs1_blocks => p_fs1_blocks,
fs2_bytes => p_fs2_bytes,
fs2_blocks => p_fs2_blocks,
fs3_bytes => p_fs3_bytes,
fs3_blocks => p_fs3_blocks,
fs4_bytes => p_fs4_bytes,
fs4_blocks => p_fs4_blocks,
full_bytes => p_full_bytes,
full_blocks => p_full_blocks,
unformatted_blocks => p_unformatted_blocks,
unformatted_bytes => p_unformatted_bytes
);
dbms_output.put_line('Full bytes = '||p_full_bytes);
dbms_output.put_line('Full blocks = '||p_full_blocks);
dbms_output.put_line('UF bytes = '||p_unformatted_bytes);
dbms_output.put_line('UF blocks = '||p_unformatted_blocks);
end;
/
```

In this PL/SQL, you need to modify the code so that it reports on the LOB segment in your environment.

Secure Files Space Used

Here is an example of how to call `DBMS_SPACE.SPACE_USAGE` for a Secure Files

LOB:

```
SQL> DECLARE
    l_segment_owner varchar2(40);
    l_table_name varchar2(40);
    l_segment_name varchar2(40);
    l_segment_size_blocks number;
    l_segment_size_bytes number;
    l_used_blocks number;
    l_used_bytes number;
    l_expired_blocks number;
    l_expired_bytes number;
    l_unexpired_blocks number;
    l_unexpired_bytes number;
    --
    CURSOR c1 IS
        SELECT owner, table_name, segment_name
        FROM dba_lobs
        WHERE table_name = 'PATCHMAIN';
BEGIN
    FOR r1 IN c1 LOOP
        l_segment_owner := r1.owner;
        l_table_name := r1.table_name;
        l_segment_name := r1.segment_name;
        --
        dbms_output.put_line('-----');
        dbms_output.put_line('Table Name : ' || l_table_name);
        dbms_output.put_line('Segment Name : ' || l_segment_name);
        --
        dbms_space.space_usage(
            segment_owner => l_segment_owner,
            segment_name => l_segment_name,
            segment_type => 'LOB',
            partition_name => NULL,
            segment_size_blocks => l_segment_size_blocks,
            segment_size_bytes => l_segment_size_bytes,
            used_blocks => l_used_blocks,
            used_bytes => l_used_bytes,
            expired_blocks => l_expired_blocks,
            expired_bytes => l_expired_bytes,
```

```
unexpired_blocks => l_unexpired_blocks,
unexpired_bytes => l_unexpired_bytes
);
--
dbms_output.put_line('segment_size_blocks: '||l_segment_size_blocks);
dbms_output.put_line('segment_size_bytes: '||l_segment_size_bytes);
dbms_output.put_line('used_blocks : '|| l_used_blocks);
dbms_output.put_line('used_bytes : '|| l_used_bytes);
dbms_output.put_line('expired_blocks : '|| l_expired_blocks);
dbms_output.put_line('expired_bytes : '|| l_expired_bytes);
dbms_output.put_line('unexpired_blocks      :      '||
l_unexpired_blocks);
dbms_output.put_line('unexpired_bytes : '|| l_unexpired_bytes);
END LOOP;
END;
/
```

Again, in this PL/SQL, you need to modify the code so that it reports on the table with the LOB segment in your environment.

7. Reading BFILEs:

As discussed previously, a BFILE data type is simply a column in a table that stores a pointer to an OS file. A BFILE provides you with read-only access to a binary file on disk. To access a BFILE, you must first create a directory object. This is a database object that stores the location of an OS directory. The directory object makes Oracle aware of the BFILE location on disk.

Example:

In this example we will create first a directory object, then create a table with a BFILE column, and then we will use the `DBMS_LOB` package to access a binary file:

```
SQL> create or replace directory load_lob as  
'/orahome/oracle/lob';
```

Next, a table is created that contains a BFILE data type:

```
SQL> create table patchmain  
(patch_id number  
,patch_file bfile);
```

For this example, a file named `patch.zip` is located in the aforementioned directory. You make Oracle aware of the binary file by inserting a record into the table using the directory object and the file name:

```
SQL> insert into patchmain values(1,  
bfilename('LOAD_LOB','patch.zip'));
```

Now, you can access the BFILE via the `DBMS_LOB` package. For instance, if you want to verify that the file exists or display the length of the LOB, you can do so as follows:

```
SQL>select  
dbms_lob.fileexists(bfilename('LOAD_LOB','patch.zip')) from  
dual;  
SQL> select dbms_lob.getlength(patch_file) from patchmain;
```

In this manner, the binary file behaves like a BLOB. The big difference is that the binary file is not stored within the database.

8. Quiz:

1. What is the wrong statement related to supported LOB types?
 - a) Character large object (CLOB)
 - b) Date large object (DLOB)
 - c) Binary large object (BLOB)
 - d) Binary file (BFILE)
 - e) National character large object (NCLOB)
2. Which is the correct statements about LOB types that stores character documents, such as big text files, log files, XML files, and so on?
 - a) BLOB
 - b) NCLOB
 - c) CLOB
 - d) BFILE
3. Which is the correct statements about LOB types that stores data in national character set format; supports characters with varying widths?
 - a) BLOB
 - b) NCLOB
 - c) CLOB
 - d) BFILE
4. Which is the correct statements about LOB types that stores unstructured bitstream data (images, video, and so on)?
 - a) BLOB
 - b) NCLOB
 - c) CLOB
 - d) BFILE

5. LOBs are a data type that is suited to storing large and unstructured data.
- a) True
 - b) False
6. Which is the correct statements about LOB types that stores on the file system outside of database; read-only?
- a) BLOB
 - b) NCLOB
 - c) CLOB
 - d) BFILE
7. BFILEs participate in database transactions and can be backed up, restored, and recovered by Oracle. In other words, all the Atomicity Consistency Isolation Durability (ACID) properties
- a) True
 - b) False
8. Which LOBs do participate in database transactions and can be backed up, restored, and recovered by Oracle?
- a) BLOB
 - b) All the other options
 - c) BFILE
9. Which LOBs do not participate in database transactions and cannot be backed up, restored, and recovered by Oracle?
- a) BLOB
 - b) All the other options
 - c) BFILE

10. What is the ideal option for storing and manipulating semi-structured data include document files such as json, XML documents or word processor, and log files?
- a) BLOB
 - b) CLOB only
 - c) CLOB and NCLOB
 - d) BFILE
11. Which is the correct statement about LOBs where locators are stored in rows containing a LOB column?
- a) Internal LOBs
 - b) Internal and External LOBS
 - c) External LOBs
 - d) None of the other options
12. Which is the correct statement about LOBs where the LOB locator for a BFILE stores the directory path and file name on the OS?
- a) Internal LOBs
 - b) Internal LOBs and External LOBs
 - c) External LOBs
 - d) None of the other options
13. BLOBs participate in database transactions and can be backed up, restored, and recovered by Oracle. In other words, all the Atomicity Consistency Isolation Durability (ACID) properties
- a) True
 - b) False

14. What is the default type of LOB that should be used as the way to store LOBs?

- a) BasicFiles
- b) None of the other options
- c) Both BasicFiles and SecureFiles
- d) SecureFiles

15. What is the true statement about SecureFiles' enhancements over BasicFiles LOBs?

- a) Encryption and Compression and Deduplication
- b) Compression only
- c) Deduplication only
- d) Encryption and Compression only

16. What is the meaning of the deduplication feature of SecureFiles LOBs?

- a) Eliminates extra data that LOBs contains over 6GB.
- b) Eliminates extra data that LOBs contains over 2GB.
- c) Eliminates duplicate LOBs that otherwise would be stored multiple times.
- d) Duplicates LOBs that otherwise would be stored multiple times.

17. What is the command you can use to modify the value of DB_SECUREFILE parameter?

- a) ALTER SYSTEM only
- b) Either ALTER SYSTEM or ALTER SESSION
- c) ALTER SESSION only
- d) The value of DB_SECUREFILE cannot be changed

18. Oracle creates a LOB segment and a LOB index for each LOB column?

- a) True
- b) False

19. Choose the correct statement that describe the implemented of partitioned LOB?

- a) Storage options are the same for all partitions
- b) No partitions for a LOB at all
- c) Only one partition is allowed for a LOB
- d) Each LOB partition is created with its own storage options

20. When a table is queried, the database uses the LOB locator and associated LOB index to locate the appropriate LOB chunks.

- a) True
- b) False

Quiz answers – chapter 11	
1	b
2	c
3	b
4	a
5	a
6	d
7	b
8	a
9	c
10	c
11	a
12	c
13	a
14	d
15	a
16	c
17	b
18	a
19	d
20	a

References:

Books

1. Oracle Database Administrator's Guide, 18c (Oracle University), March 2019, Primary Author: Rajesh Bhatiya, Randy Urbano
2. database-new-features-guide (Oracle University), 18c E88909-01 February 2018, Primary Authors: Tanaya Bhattacharjee, Sunil Surabhi, Mark Baue
3. The Cloud DBA-Oracle: Managing Oracle Database in the Cloud – First Edition, Abhinivesh Jain and Niraj Mahajan. Apress – ISBN-13 (pbk): 978-1-4842-2634-6 ISBN-13 (electronic): 978-1-4842-2635-3

Web Sites

1. <https://docs.oracle.com/database>
2. <https://docs.oracle.com/en/database/oracle/oracle-database/19/whats-new.html>
3. <http://www.oracle.com/technetwork/database/enterprise-edition/downloads/index.html>